3

SESSION 3

Object-Oriented Dart

90 minutes

- Quick recap of Sessions 1 & 2
- Classes, objects & constructors
- Named & factory constructors, initializer lists
- Getters, setters & inheritance

53

SESSION 3 AGENDA

What We'll Cover Today

1. Quick recap of Sessions 1 & 2
2. Classes, objects & constructors
3. Named & factory constructors, initializer lists
4. Getters, setters & inheritance
5. Abstract classes, interfaces & mixins
6. Enums, extension methods & operator overloading
7. Static members, immutability & const constructors
8. Hands-on build & wrap-up

54

RECAP

Quick Recap: Sessions 1 & 2

- Null safety: String vs String?, with ?., ??, and !
- Named parameters + arrow functions shape every Flutter callback
- map / where / reduce / fold operate on List, Map & Set
- try/catch/finally and custom exceptions handle failure

WHY OOP

Why Classes?

- Data represents the real world; classes model “things”
- Rule of thumb: “When in doubt, make a new class”
- A class is a blueprint; an object is one instance of it
- Nearly all Flutter code lives inside classes — including every widget

2.5.1 CLASSES

Classes & Objects Basics

```
class Cat {
  String name;
  String color;
}
var cat = Cat();
cat.name = 'Nora';
```

- class defines the fields (data) and methods (behavior)
- Dart 2+ doesn't need the new keyword to instantiate
- Each object has its own independent copy of the fields

57

2.5.2 CONSTRUCTORS

Constructors & this. Shorthand

```
class Animal {
  String name, type;
  Animal(this.name, this.type);  // shorthand
}
var dog = Animal('Rex', 'Dog');
```

- A constructor shares the class's name and runs on creation
- this.name auto-assigns a constructor argument to the field
- Constructor bodies can run extra logic, not just assignment

58

2.5.4 NAMED CONSTRUCTORS

Named Constructors

```dart
class Point {
  double x, y;
  Point(this.x, this.y);
  Point.origin() : x = 0, y = 0;   // named ctor
}
var p = Point.origin();  // (0, 0)
```

What does the colon (:) do?

It starts the initializer list.

Dart runs it before the constructor body.

Here it sets the fields directly:

`x = 0, y = 0`

- ClassName.constructorName gives a class multiple ways to build
- Always returns a brand-new instance of that class
- Common examples: List.filled(), DateTime.now()

59

2.5.4 NAMED CONSTRUCTORS

More Named Constructors

```dart
class User {
  final String name;
  final bool isGuest;

  User(this.name) : isGuest = false;
  User.guest()
      : name = 'Guest', isGuest = true;
}
var u = User.guest();  // Guest
```

```dart
class Circle {
  final double radius;

  Circle(this.radius);
  Circle.fromDiameter(double d)
      : radius = d / 2;
}
var c = Circle.fromDiameter(10);  // 5
```

- Give each natural way of creating an object its own name
- The name says what the call does: guest, fromDiameter, origin
- Great for default values and converting from other units or formats

60

2.5.4 NAMED CONSTRUCTORS

Named Constructors: fromJson

```
class Product {
  final String title;
  final double price;

  Product({required this.title, required this.price});

  Product.fromJson(Map<String, dynamic> json)
      : title = json['title'],
        price = (json['price'] as num).toDouble();
}
var p = Product.fromJson({'title': 'Shoes', 'price': 49.99});
```

- APIs and Firebase return JSON; fromJson converts it into a typed object
- The UI reads product.title, not json['title'], so typos are caught early
- Pair it with toJson() to send data back to the server

61

PRACTICE

Quick Exercise

```
class Temperature {
  final double celsius;
  Temperature(this.celsius);
  // TODO: add Temperature.fromFahrenheit(double f)
}
var t = Temperature.fromFahrenheit(212);  // 100.0 °C
```

- Formula: °C = (°F − 32) × 5 / 9
- Use the colon initializer list to set the final field
- Stretch: add Temperature.freezing() and Temperature.boiling()

62

2.5.4 FACTORY CONSTRUCTORS

Factory Constructors

```dart
class Logger {
  static Logger? _instance;
  Logger._internal();  // private named ctor
  factory Logger() {
    return _instance ??= Logger._internal();
  }
}
```

- factory can return a cached instance instead of always creating new
- Enables patterns like singletons and object pools
- A leading underscore (_) makes a name private to its library/file

63

EXTRA

Initializer Lists & Assertions

```dart
class Circle {
  final double radius;
  final double area;
  Circle(this.radius)
      : area = 3.14159 * radius * radius,
        assert(radius > 0, 'radius must be positive');
}
```

- The colon (:) section runs before the constructor body
- Perfect for computing final fields from constructor args
- assert() catches programmer mistakes in debug builds only

64

EXTRA

Getters and Setters

```dart
class Rectangle {
  double _width, _height;
  Rectangle(this._width, this._height);
  double get area => _width * _height;
  set width(double w) { _width = w < 0 ? 0 : w; }
}
```

- get exposes a computed, read-only property — no ( ) needed to call it
- set intercepts assignment, letting you validate or transform
- Widely used to expose derived data without storing it redundantly